innFactory

Anwendung der funktionalen Programmierung mit Scala

Intro & Organisatorisches
Gliederung

- 1. Intro & Organisatorisches
- 2. Funktionale Programmierung
- 3. Lambda-Kalkül
- 4. Einführung in Scala
- 5. Anleitung Scala Setup

Intro & Organisatorisches
Wer sind wir?

Nicolas Schlecker Bereichsleiter | Full-Stack Engineer
Bachelor Informatik TH Rosenheim
E-Mail: n.schlecker@innfactory.de

Patrick Stadler Full-Stack Engineer
Bachelor Informatik TH Rosenheim
E-Mail: p.stadler@innfactory.de

Intro & Organisatorisches
Lernziele der Vorlesung

- Konzepte und typische Merkmale des funktionalen Programmierens **kennen, verstehen und anwenden** können:
  - Modellierung mit algebraischen Datentypen
  - Rekursion & Endrekursion
  - Starke Typisierung & Typinferenz
  - Funktionen höherer Ordnung (map, filter, fold)
  - Monadische Fehlerbehandlung (Option, Either)
- Datenstrukturen und Algorithmen in Scala umsetzen und auf praktische Probleme anwenden können
- Funktionale Architekturen (Aktoren, Event Sourcing, Web, Big Data) einordnen und einsetzen können

Intro & Organisatorisches
Prüfungsleistung

- **Vorträge**
  - Teams aus je 2 Personen
  - Themenwahl wie FWPM-Wahl
  - 25–30 Minuten
  - Teil der Gesamtnote

- **Mündliche Prüfung**
  - Am 09.07.2026 (Letzter Vorlesungstermin)
  - 15 Minuten

Intro & Organisatorisches
Vortragsthemen

- Lisp & Scheme
- Haskell
- Funktionale Programmierung mit Python
- Clojure: FP auf der JVM
- Funktionale Konzepte in JavaScript (fp-ts / effect-ts)
- Erlang und Elixir
- Funktionale Konzepte in Rust
- F#
- Scala Framework: Spark Streaming
- Scala Framework: ZIO
- Scala Framework: Spotify SCIO
- Scala Typelevel Ökosystem

Funktionale Programmierung

Funktionale Programmierung

Was versteht man unter funktionaler Programmierung?

- Ein **Programmierparadigma** neben Imperativ und Objektorientiert
- **Funktionen** stehen im Zentrum:
  - Übergabe von Funktionen als Parameter
  - Funktionen als Rückgabewert anderer Funktionen
  - Kombination/Komposition von Funktionen
- Funktionale Programme beschreiben die Lösung durch **verschachtelte Funktionsaufrufe und Ausdrücke,** nicht durch Schritt-für-Schritt-Anweisungen

Funktionale Programmierung
Wer benutzt funktionale Programmierung?

- **GitHub / Semantic** | Parsing, Analyzing Source Code (Haskell)
- **NuBank** | Brasilianische Bank, Microservices mit Clojure
- **Disney+** | Echtzeit-Datenverarbeitung (Scala)
- **Twitter/X** | Finagle für high-concurrency RPC-Systeme (Scala)
- **Airbnb** | Recommendation Engine, Aerosolve ML Framework (Scala)
- **Spotify** | SCIO Framework (Scala API für Apache Beam), Spotify Wrapped
- **innFactory** | Play Framework, Pekko, smithy4s (Scala)

Funktionale Programmierung
Anwendungsgebiete

- **Datenverarbeitung & Big Data** | Apache Spark, Flink, Kafka Streams
- **Webentwicklung** | Play Framework, Pekko HTTP, ZIO HTTP, http4s
- **Verteilte Systeme & Microservices** | Pekko, ZIO
- **Nebenläufige Anwendungen** | Cats Effect, ZIO
- **Wissenschaftliche Berechnungen** | Breeze, Spire

Funktionale Programmierung
Paradigmenvergleich: Imperativ

- **Imperativ (z.B. C)**
  - Beschreibt **wie** ein Problem gelöst wird, Schritt für Schritt
  - Abfolge von Zustandsänderungen (Zuweisungen, Schleifen, etc.)
  - Programmierer gibt genaue Anweisungen
  - Mutable State: Variablen werden verändert

```
// Beispiel: Summe der geraden Zahlen von 1-10
int sum = 0;
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) {
        sum += i;  // Zustand wird verändert
    }
}
```

Funktionale Programmierung
Paradigmenvergleich: Objektorientiert (OOP)

- **Objektorientiert (z.B. Java)**
  - Organisiert Code um **Objekte**, die Zustand und Verhalten kapseln
  - Fokus auf Datenstrukturen (Klassen) und deren Beziehungen
  - Kapselung: Zustände in Objekten verborgen, Interaktion über Methoden
  - OOP ist letztlich auch imperativ, jedoch mit zusätzlicher Struktur

```
// Beispiel: Objekt kapselt Zustand + Verhalten
class Counter {
    private int count = 0;
    public void increment() { count++; }  // Mutation!
    public int getCount() { return count; }
}
```

Funktionale Programmierung
Paradigmenvergleich: Funktional (FP)

- **Funktional (z.B. Scala, Haskell)**
  - Beschreibt **was** berechnet werden soll, nicht wie
  - Vermeidet veränderbare Zustände und Seiteneffekte: **Immutability**
  - Programme bestehen aus **Ausdrücken** (Expressions), nicht aus Anweisungen
  - Deklarativ: Man beschreibt Eigenschaften des Ergebnisses

```scala
// Beispiel: Summe der geraden Zahlen von 1-10
val sum = (1 to 10).filter(_ % 2 == 0).sum
// Kein veränderter Zustand, kein Loop, kein var
```

Funktionale Programmierung
Expressions vs. Statements

- **Statement** (Anweisung): Führt eine Aktion aus, hat **keinen Rückgabewert**
  - x = 5; → Zuweisung, kein Wert
  - for (...) { ... } → Schleife, kein Wert
  - System.out.println("Hi") → Seiteneffekt, kein nützlicher Wert

- **Expression** (Ausdruck): Wird zu einem **Wert ausgewertet**
  - 5 + 3 → ergibt 8
  - if (x > 0) "positiv" else "negativ" → ergibt einen String
  - list.map(_ * 2) → ergibt eine neue Liste

- In der FP ist **alles ein Ausdruck**. Auch if/else, match, Blöcke → alles hat einen Rückgabewert.

Funktionale Programmierung
Warum FP? Einfachheit und Klarheit

- Keine versteckten Zustände und Seiteneffekte
- Funktionen verhalten sich wie **mathematische Funktionen**: gleicher Input → gleicher Output
- Leichter verständlicher und kürzerer Code
- Einfacheres Debugging und bessere Nachvollziehbarkeit

Funktionale Programmierung
Warum FP? Modularität und Wiederverwendbarkeit

- Kleine, universelle Funktionsbausteine (wie Lego)
- Keine versteckten Nebeneffekte erleichtern Kombination
- Unabhängigere Entwicklung, Testbarkeit und vielseitige Wiederverwendung
- Höhere Modularität und bessere Code-Struktur

Funktionale Programmierung
Warum FP? Nebenläufigkeit und Parallelität

- Einfachere parallele Ausführung durch **Immutability** (Unveränderbarkeit)
- Reduzierung von Race Conditions und Deadlocks
- Weniger Synchronisation nötig → bessere Performance
- Besonders relevant für: Multi-Core, verteilte Systeme, Big Data

Funktionale Programmierung
Zentrale Begriffe

- **First-Class Functions:** Funktionen haben gleichen Status wie Zahlen oder Strings
  - Zuweisung an Variablen, Übergabe als Parameter, Rückgabe aus Funktionen
- **Pure Functions** (reine Funktionen)
  - Keine Seiteneffekte
  - Gleiche Eingabe → Gleiches Ergebnis (referentielle Transparenz)
  - Keine Veränderung von Daten außerhalb des lokalen Kontextes
- **Immutability** | Datenstrukturen werden nicht verändert, sondern neue aus alten abgeleitet

Funktionale Programmierung
Seiteneffekte (Impurity)

- **Was ist ein Seiteneffekt?**
- Jede beobachtbare Auswirkung einer Funktion **über den Rückgabewert hinaus**:
  - Datei lesen/schreiben (I/O)
  - Datenbank-Zugriff
  - Globale Variable verändern
  - Konsolenausgabe
  - Netzwerk-Request

- In der FP: I/O ist trotzdem möglich, aber **kontrolliert** (z.B. über IO-Monaden)
- **Reine Funktionen** haben keine Seiteneffekte → leichter testbar, vorhersagbar, parallelisierbar

Lambda-Kalkül

Lambda-Kalkül
Ursprung

- **Alonzo Church** | 1934: Lambda-Kalkül → Programmiersprachen
- **Alan Turing** | 1936: Turingmaschine → Computer

- Beide Modelle sind **gleich mächtig** (Church-Turing-These)

- Lambda-Kalkül: eine "Programmiersprache auf Papier"
- Entwickelt um logische Probleme (wie das Entscheidungsproblem) zu untersuchen

Lambda-Kalkül
Grundlagen

- Das Lambda-Kalkül besteht aus nur **drei Bausteinen:**
- **1. Variablen** sind Platzhalter für Werte/Eingaben: x, y, z
- **2. Abstraktionen** (Funktionsdefinition)
  - λx. x + 1 → "eine Funktion, die 1 zu x addiert"
  - Das λ (Lambda) markiert den Parameter (Variable wird „gebunden“) => Hier λx
- **3. Applikationen** (Funktionsaufruf)
  - (λx. x + 1) 2 → ergibt 3
  - Der Ausdruck 2 wird für x eingesetzt

Das ist alles! Aus diesen drei Bausteinen lässt sich **jede berechenbare Funktion** beschreiben.

Lambda-Kalkül
β-Reduktion

- **β-Reduktion** = das Einsetzen des Arguments in den Funktionskörper
- **Schritt für Schritt:**

  - (λx. x + 1) 5
    - → x kommt im Körper vor als [x] + 1
    - → Setze 5 für [x] ein → [5] + 1 → 6

  - (λx. x * x) 3
    - → x kommt im Körper vor als [x] * [x]
    - → Setze 3 für [x] ein → [3] * [3] → 9

Das Rechnen im Lambda-Kalkül besteht aus dem **schrittweisen Ersetzen** von Lambda-Ausdrücken.

Lambda-Kalkül
α-Konversion

- **α-Konversion |** Der Name eines Parameters ist egal, nur seine Rolle zählt.
- **Formale Regel**: $\lambda x.\ M \rightarrow \alpha\ \lambda y.\ M\ [x := y]$
  wobei y nicht frei in M vorkommt und y dort nicht gebunden ist, wo es ein x ersetzt.

  Beispiel: Da die Regel auf jeden Teilausdruck anwendbar ist, gilt:
  - $\lambda x.\ (\lambda x.\ x)\ x \equiv \alpha\ \lambda y.\ (\lambda x.\ x)\ y$
  - Nur das äußere x wird umbenannt, das innere $\lambda x.\ x$ bleibt unberührt, da es einen eigenen Bindungsbereich hat.

- **Warum wichtig?** Vermeidet Namenskollisionen bei β-Reduktion
  - $(\lambda x.\ \lambda y.\ x + y)\ y \rightarrow$ **naiv**: $\lambda y.\ y + y \leftarrow$ falsch! (Variable eingefangen)
  - Erst α-konvertieren: $\lambda y.\ x + y \rightarrow \alpha\ \lambda z.\ x + z$
  - Dann β-reduzieren: $(\lambda x.\ \lambda z.\ x + z)\ y \rightarrow \lambda z.\ y + z \leftarrow$ korrekt!

Lambda-Kalkül
Beispiele

- **Identitätsfunktion:** λx. x → gibt das Argument unverändert zurück
  - (λx. x) 5 → 5

- **Konstante Funktion:** λx. 42 → ignoriert das Argument
  - (λx. 42) 7 → 42

- **Funktionen als Argumente:**
  - (λf. f 10) (λx. x * 2)
    - → Setze (λx. x * 2) für f ein
    - → (λx. x * *2) 10* → *10* * 2 → 20

Lambda-Kalkül
Mehrere Parameter | Currying

Problem: Lambda-Kalkül kennt nur **einstellige** Funktionen

- **Lösung: Currying** – Mehrstellige Funktionen als Kette einstelliger Funktionen
  - Statt λ(x, y). x + y schreibt man: λx. λy. x + y

- **Auswertung Schritt für Schritt:**
  - (λx. λy. x + y) 3 5
    - → Setze 3 für x ein: (λy. 3 + y) 5 ← **Neue Funktion!**
    - → Setze 5 für y ein: 3 + 5 → 8

- Nach dem ersten Einsetzen erhalten wir eine **neue Funktion** λy. 3 + y, die "sich merkt", dass x = 3 war. **Das ist Currying!**

Lambda-Kalkül
Funktionskomposition

- **Funktionskomposition** – Funktionen hintereinander ausführen
  - compose = λf. λg. λx. f (g x)
  - Erst g auf x anwenden, dann f auf das Ergebnis
- **Beispiel:**
  - f = λy. y * y (quadrieren)
  - g = λz. z + 1 (um 1 erhöhen)
  - compose f g 3
    - → f (g 3) → f (3 + 1) = f 4 → 4 * 4 = 16

Funktionskomposition ist **das zentrale Prinzip** der FP: kleine Funktionen zu größeren zusammenbauen.

Lambda-Kalkül
Historische Entwicklung funktionaler Sprachen

- **LISP** (1958) | erste funktionale Sprache, inspiriert vom Lambda-Kalkül
- **ML / Standard ML** (1970er/90er) | Typinferenz, Pattern Matching
- **Haskell** (1990) | rein funktional, lazy evaluation
- **Scala** (2004) | FP + OOP auf der JVM, von Martin Odersky
- **F#** (2005) | FP auf .NET
- **Clojure** (2007) | modernes Lisp auf der JVM

Alle diese Sprachen haben ihre Wurzeln im Lambda-Kalkül von 1934!

innFactory

Scala | Vom Lambda-Kalkül zum Code

Scala Überblick

- **Hybride Sprache** | unterstützt FP und OOP (Multiparadigma)
- Entwickelt von **Martin Odersky**, erstmals erschienen 2004
- Läuft auf der **JVM** und bietet volle Interoperabilität mit Java
- Auch als **Scala.js** (JavaScript) und **Scala Native** verfügbar
- **Statisches Typsystem** mit Typ-Inferenz durch den Compiler

Scala Syntax

Variablen / Konstanten

```
// Konstante
val x: Int = 0 (Der Typ ist hier optional)

// Variable
var x: Int = 0 (Der Typ ist hier optional)
```

Scala Syntax

Funktionen

```
// Funktionsdefinition mit def
// Kein "return" nötig, letzter Ausdruck = Rückgabewert
def plusOne(i: Int): Int = i + 1(Rückgabe-Typ ist hier optional)
```

Scala Syntax

innFactory

Klassen und Objekte

```
// Klassendefinition
class MyClass(intArgument: Int, …) {
   // body
}

// Singleton-Objekt, welches automatisch und nur einmal instanziiert wird.
object MyObject {}

// Selbe Instanz / Referenz
val obj1 = MyObject
val obj2 = MyObject
```

Scala Syntax

innFactory

Klassen und Objekte

```
// Klasse, für welches ein „Companion-Object“ mit apply Methode generiert wird.
case class MyCaseClass(attr: Int) {
   // body
}

// Automatisch / Nicht sichtbar generiert
object MyCaseClass {
    def apply(attr: Int) = new MyCaseClass(attr)
}

val x = MyCaseClass(5)
```

Einführung in Scala
Variablen | val vs. var

- **Faustregel:** Immer val verwenden. var nur in Ausnahmefällen.
- **Bezug zum Lambda-Kalkül:** Im Lambda-Kalkül gibt es keine Zuweisungen, keine Mutation. val bildet genau das ab.

```
// val = unveränderlich (immutable) → STANDARD in FP!
val x = 6
// x = 5  → Compiler Error!

// var = veränderlich (mutable) → vermeiden in FP
var y = 6
y = 5  // OK, aber: in FP versuchen wir var zu vermeiden!
```

Einführung in Scala
Funktionen definieren

- **Bezug zum Lambda-Kalkül:**
  - (x: Int) => x + *2 ist das Scala-Äquivalent zu λx. x* + 2

```
def square(x: Double): Double = x * x
def add(a: Int, b: Int): Int = a + b
```

Funktionen sind Typen, können anonym erstellt, zugewiesen und übergeben werden!

```
val plusTwo: Int => Int = (i: Int) => i + 2
def addFive(f: Int => Int): Int = f(2) + 5

val result = addFive(plusTwo)
// result = (2 + 2) + 5 => 9
```

Lambda-Kalkül und Scala (Direktvergleich)

- λx. x → (x: Int) => x (Identitätsfunktion)
- λx. 42 → (_: Int) => 42 (Konstante Funktion, Input egal)
- λx. x + 1 → (x: Int) => x + 1 (Nachfolger)

Einführung in Scala
Lambda-Kalkül und Scala (Direktvergleich)

- λf. f 10 → (f: Int => Int) => f(10) (Funktion als Argument)
- λx. λy. x + y → (x: Int) => (y: Int) => x + y (Currying!)
- (λx. x + 1) 5 → 6 → ((x: Int) => x + 1)(5) → 6 (β-Reduktion)
- **Int => Int** ist ein Funktionstyp: "Nimmt ein Int und gibt ein Int zurück.“

Die Verbindung zwischen Lambda-Kalkül und Scala ist **direkt und unmittelbar**!

Lambda-Kalkül und Scala (Direktvergleich)

Alpha Konversion / Scoping / Shadowing und freie Variablen

```
// Freie Variable y hat den Wert 5
val y = 10

// Funktion mit Parameter y (gebunden) – shadowed das äußere y
val add = (x: Int) => (y: Int) => x + y
val result = add(y)   // add(10)
// result = (y: Int) => 10 + y
result(3)  // → 13, NICHT 3 + 3 = 6
```

Wenn der Compiler kein Shadowing machen würde, könnte add(y) fälschlicherweise (y: Int) => y + y ergeben.
Das äußere y=10 ginge verloren und das Ergebnis hinge nur noch vom Parameter ab.
Der Compiler verhindert genau das. (Keine direkte Alpha Konversion im Scala Compiler)

Java vs. Scala | Alle geraden Zahlen verdoppeln

```java
// Java (imperativ)
List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
List<Integer> result = new ArrayList<>();
for (int n : numbers) {
    if (n % 2 == 0) { result.add(n * 2); }
}
```

```scala
// Scala (funktional)
val numbers = List(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
val result = numbers.filter(_ % 2 == 0).map(_ * 2)
// result = List(4, 8, 12, 16, 20)
```

Java vs. Scala | Alle geraden Zahlen verdoppeln

- **Beobachtungen:**
  - Scala: kein var, keine Schleife, kein manuelles Hinzufügen
  - filter und map sind **Higher-Order Functions**
  - **_ % 2 == 0** ist ein **Lambda-Ausdruck** (Kurzschreibweise und äquivalent zu: **v => v % 2 == 0**)

```
// Scala (funktional)
val numbers = List(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
val result = numbers.filter(_ % 2 == 0).map(_ * 2)
// result = List(4, 8, 12, 16, 20)
```

Einführung in Scala
if als Expression

- In Scala ist if/else ein **Ausdruck**, der einen Wert zurückgibt. Kein var nötig!

```
// Java – if ist ein Statement
String label;
if (score > 90) {
    label = "sehr gut";
} else {
    label = "ok";
}

// Scala – if ist ein Ausdruck (Expression)
val label = if score > 90 then "sehr gut" else "ok"
```

Scala Setup (für nächste Woche) | Linux und Mac

- **IntelliJ IDEA** Lizenz z.B. über GitHub Student Developer Pack (nicht Community Edition)
- **Scala Plugin** für IntelliJ installieren, direkt über IntelliJ UI
- **sbt** (Scala Build Tool) wird mit dem Plugin mitgeliefert
- **Scala Installer oder per Package Manager, z.B. brew**
- **JDK 17+** z.B. über SDKMAN, Adoptium oder direkt über die IntelliJ UI
- **Docker Desktop** für spätere Übungen

Scala Setup (für nächste Woche) | Windows

- **IntelliJ IDEA** Lizenz z.B. über GitHub Student Developer Pack (nicht Community Edition)
- **Scala Plugin** für IntelliJ installieren, direkt über IntelliJ UI
- **sbt** (Scala Build Tool) wird mit dem Plugin mitgeliefert
- **Scala Installer**
- **JDK 17+** z.B. über IntelliJ UI
- **Docker Desktop** für spätere Übungen

Scala Setup | Links

- **IntelliJ IDEA** (www.jetbrains.com/idea)
- **Scala** (www.scala-lang.org/download/)
- **IntelliJ Plugin Guide** (https://www.jetbrains.com/help/idea/managing-plugins.html)
- **sbt** (https://www.scala-sbt.org)
- **Scala Installer (**https://www.scala-lang.org/download/)
- **SDKMAN** (sdkman.io), Adoptium (adoptium.net)
- **IntelliJ UI JDK Download** (www.jetbrains.com/guide/java/tips/download-jdk/)
- **Docker** (https://docs.docker.com/desktop/)

Einführung in Scala
Scala Setup | Validierung

- **IntelliJ Testprojekt aus Github öffen und ausführen**
  Vorlesung 1 unter https://github.com/innFactory-Classrooms/afps/tree/main/VL-01

REPL ausprobieren:

```
# Terminal öffnen, in den Projektordner navigieren
sbt console       # Startet die Scala-REPL
# In der REPL:
scala> val x = 42
scala> (x: Int) => x + 1
# Beenden mit:
scala> :quit
```